Title

AUTHORS

1 Introduction

Digital money must be a strict improvement over physical cash: it must be free to use, open to anyone, and entirely under your control. And yet when we move payments online, we lose control of our financial resources, gatekeepers restrict access to payment infrastructure, and, even though it is vastly easier to send digital messages than physical ones, every transaction gets a small embedded fee.

This is an absurd state of affairs.

Blockchains promised to fix this. Decentralization and replication mean that a blockchain keeps running even when nodes suffer hardware failure or regional networks experience outages. Smart contracts let anyone upload code governing the movement of their assets, and immediately know that their rules will be followed by the global system. The only requirement to participate in all of this is a network connection and a device capable of calculating a cryptographic signature, which any new smartphone can handle easily.

And yet, today's blockchains are slow and expensive to use. Blockchains are forced to charge increasingly high fees as demand to use them increases, because they cannot easily scale up the volume of transactions they can handle. This limitation is not a question of just improving the engineering, but is a fundamental limitation of their architectures.

Our mission, with Project Thruput, is to eliminate every practical scaling limit, so that we can make smart contract execution free.

1.1 The Curse of Ordering

The root of the problem is that blockchains pretend to execute transactions sequentially. This is a natural design choice, and makes it easy to ensure that smart contract execution produces the same results on every node. Developers are used to writing sequential, "single-threaded" code.

Yet there is only so much sequential computation that can be done within one unit of time. We cannot simply buy a faster CPU core today. The common choice is to subdivide a unit of time into more, shorter slices, but this limits the complexity of smart contracts and pushes developers into arcane tricks to hyperoptimize smart contract computation. cite solana assembly sdk Adding CPU cores can parallelize transactions that touch disjoint state, but when one transaction depends on the output of another, then ultimately the second transaction has to be run after the first. Yet shared state is central to the financial markets that blockchains enable. Blockchains regularly see many users transacting on the same market, or many buyers paying one merchant, or moving assets through the same DeFi protocol.

Additional CPUs fundamentally cannot provide more than a bounded speedup on these ordered workloads. Increasing transaction throughput under an ordering constraint, therefore, means a series of increasingly sophisticated engineering efforts in pursuit of ever-diminishing marginal returns. Imposing artificial divisions on state, as in sharded blockchains and Layer-2 systems, merely hides the problem of managing shared resources (cross-shard transactions become even more expensive) and makes user experience meaningfully worse.[1].

[1]No user should need to think about what shard their money lies on

Author's Contact Information: Authors.

As a result, what a sequential blockchain delivers is a system that is expensive to use, can barely realize the full potential of smart contract customization, and cannot support concurrent usage by everyone around the world.

1.2 Project Thruput

Thruput demonstrates that this assumption, that blockchains must totally order every transaction, is false. In Thruput, blocks remain ordered relative to each other, but all transactions within one block semantically execute at exactly the same logical instant. Deterministic execution of a block follows not from ordering but from structured APIs to read and write persistent state that make concurrent transactions *commute* with each other, and blocks are packed carefully to ensure key invariants, like non-negative token balances, are preserved. Contracts encapsulate these APIs behind familiar interfaces, and can recover ordering between operations as necessary.

This design enables Thruput to scale up in the simplest way possible: just by adding additional CPUs to process transactions. Moreover, our implementation scales **linearly** to hundreds of CPU cores; doubling the number of CPU cores doubles the thruput. This design fundamentally changes the tradeoff between blockchain throughput and capacity planning for users of the chain. We do not claim any level of maximum throughput; instead, our design means one can calculate, for any given level of desired transaction throughput, how many CPUs are needed.

Furthermore, this design enables new system architecture opportunities in a Thruput node. Each node is implemented as a distributed system, with, for example, transaction execution on different machines from the database. There are a limited number of CPUs that can fit on one motherboard, so Thruput scales instead by connecting additional commodity machines. Thruput can tolerate database I/O delays because Thruput's parallelism means that while one transaction might be stalled waiting for a read from the remote database, the CPU always has more work available to do.

Through all of this, Thruput acts from the outside like a normal blockchain. Each node in the system maintains a copy of the chain. There is no sharding of state or artificial subdivision between groups of users, and no extra security requirements beyond standard honest-supermajority assumptions of consensus protocols. Thruput does not impose additional requirements, such as precalculated transaction read/write sets, on end users.

This scalability lets Thruput realize the vision of a global, open, free, and programmable financial infrastructure. Because Thruput is not bound by any fixed level of transaction throughput, Thruput can scale supply arbitrarily in response to demand, and in so doing obviate the need to charge any transaction fee whatsoever to limit demand. Extreme events in one part of the world or bursts of activity in a memecoin can never crowd out normal payment activity. There will, of course, be plenty of valuable services (which may be able to charge fees) built on top of freely-transferrable digital money, but the smart contracts themselves will be free to run.

Project Thruput is under active development. The design is loosely based on prior research by some of the authors cite, but has advanced substantially beyond that into a fully distributed system architecture and a vastly more capable system. We outline here the technical innovations that we believe will make Thruput the highest-throughput blockchain possible, describe the current state of development, and survey the technical roadmap for where Project Thruput goes next.

probably need to cite SCR somewhere

2 Thruput Smart Contract Overview

Like any blockchain, smart contract execution on Thruput must be entirely deterministic and thus replicable across every node in the system. Thruput's core innovations lie in parallelism and the tools for smart contracts to manage concurrency, so in all other aspects of the system design, we strive to make Thruput as "normal" as possible.

Smart contracts in Thruput are implemented as RISC-V programs (‘riscv32-im‘). Memory and registers in the program state are discarded after the program finishes, and contracts read and write to persistent storage via a system call interface. Persistent storage is a key-value store with 64-byte keys and values with structured types, discussed below section number. Each contract is deployed at a 32-byte “ContractAddress” and owns a 32-byte keyspace within the key-value store (so every item is indexed by ‘ContractAddress || StorageKey’). One contract can synchronously invoke another contract within the same transaction. Contracts emit events to signal actions to external systems.

What makes Thruput unique lies in how transactions interact with that key-value store. In a traditional, sequential blockchain, values written in one transaction can be read in the next transaction, and in fact maintaining coherence about what writes are visible to what reads is the core engineering challenge when optimistically executing a sequence of transactions in parallel (via techniques like Block-STM [4]). Thruput has no need for a complex concurrency management protocol. Instead, Thruput changes how transactions read and write to shared state.

Snapshot Isolation. All transactions in a Thruput block read from the same snapshot of the database, as it existed at the end of the previous block. Each transaction’s side effects cannot be observed by any transaction in the same block, and only observed by transactions in future blocks. These semantics are similar to Snapshot Isolation in traditional database designs.

Write Contention. If every transaction wrote to distinct keys, then of course Thruput would have no problem reconciling concurrent writes to produce the database snapshot for the next block. The interesting use-cases for blockchains, however, are exactly those where contention on shared data is frequent and often the intended behavior. Markets, and especially automated market-makers, sequence trade orders and offer the best price to whichever trade is executed first. NFT drops often allocate a limited number of objects to be allocated to the first buyers. Even simple payments experience contention on high-activity accounts, such as flows in and out of exchanges and banks, or payments to the tax authorities on tax day.

What Thruput does is ask contract writers for a little bit of additional information on how to resolve concurrent writes. Take a standard token contract as an example. In a standard blockchain environment, to send a payment, the contract reads the sender’s balance, subtracts from it, aborts if that balance is negative, and writes an updated balance. It then reads the receiver’s balance, adds to it, and writes an updated balance (and performs other authorization checks, executes callbacks, etc). The blockchain node executing that smart contract, however, just sees an opaque sequence of bytecode operations, along with two reads and two writes to the key-value store, and has no knowledge about the algebraic relationship between the reads and the writes.

Thruput, instead, asks contracts to surface that small amount of information to the runtime, so that the blockchain can resolve concurrent writes in a predictable manner. Instead of just writing an updated balance (which, to the runtime, looks like an opaque string of bytes), the token contract outputs to the runtime an instruction to "add X" to the value (whose type is an integer, as implied by writes to that key in prior blocks) at one key, and "subtract X" from the value at another key. Then, when multiple transactions modify the same balance (if, for example, one user sends or receives many payments at once), the runtime can add all of the additions and subtractions to the balance to compute the value for the end-of-block snapshot. This value is deterministic, regardless of the temporal, possibly interleaved order that physical CPUs executed those transactions, because addition is commutative. This commutativity is what allows Thruput to be a deterministic blockchain without suffering any of the synchronization overhead of trying to keep transaction-running CPUs executing in some predictable order.

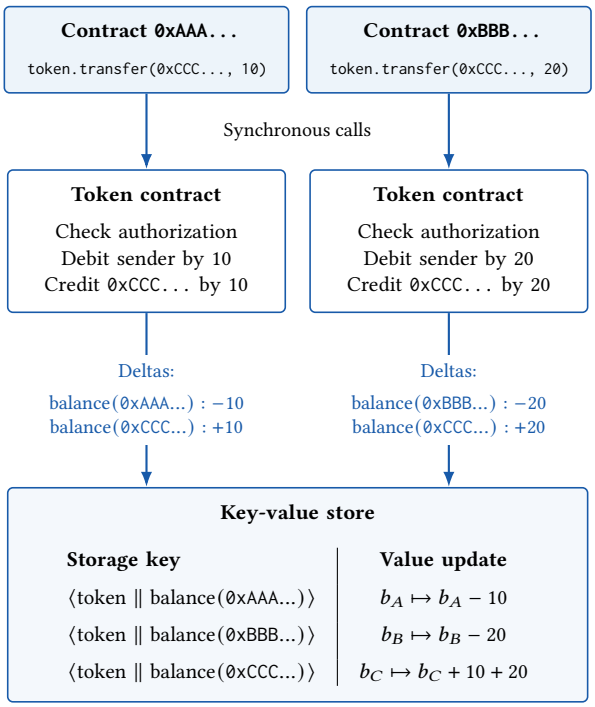


Fig. 1. **Two transactions in the same block.** The token contract wraps integer update calls in an ordinary `transfer` API, which other contracts can synchronously call. Two transactions (left and right) both modify the balance of address $0xCCC...$, and the Thruput runtime mediates the concurrent access.

Figure 1 illustrates how a token contract can encapsulate these primitives behind a familiar `transfer` API. Other contracts can use ordinary synchronous contract calls.

Commutativity is, however, insufficient to actually write useful applications on a blockchain. [2] Balances of a token, for example, cannot be allowed to become negative. Thruput therefore enforces constraints on the aggregate behavior of transactions in a block. In the case of a token balance, Thruput guarantees that after adding up all of the additions and subtractions to a balance, that integer's value is nonnegative. The blockchain manages this by being selective about including transactions when proposing a new block, deferring (nondeterministically) transactions that would violate a constraint to a future block. This block proposal process, despite the invariants to maintain, is in fact no more expensive computationally than validating and replaying a proposal from another blockchain node; see section for details.

In other words, each value type (detailed in section) in Thruput has two associated properties.

Typed Commutative Interface. Contracts read and modify values via a structured API. Each modification (like "add" to an integer) is associated with a particular type of value. These structured modifications allow the Thruput runtime to mediate writes across multiple transactions.

[2]Without these invariants, the Thruput state model would look like an application of Conflict-Free Replicated Data Types (CRDTs) citation. CRDTs cannot implement a notion of a non-negative token balance, because one cannot subtract from a balance and know that there is not some other operation that has not been applied yet to the local copy that also subtracts from a balance. This notion of conflict-freeness is stronger than Thruput needs, because Thruput need only manage potential semantic conflicts between a finite set of transactions in one block.

Value Invariants. The Thruput runtime maintains specific invariants (defined by the value type) on modifications to a value by transactions in the same block. These prevent the kinds of semantic conflicts, like a balance going negative, that cannot be addressed within one transaction, in Thruput's parallel state model.

These tools are optional for contract developers. Thruput does not force concurrency onto contracts, but gives contracts the tools to manage it should they choose. A contract writer could choose to ignore them (and use only a basic "bytestring" type), at which point Thruput would not have information about how to resolve concurrent writes and would have to exclude all but one transaction on a contract per block, in the worst case. In the other extreme, contracts using these tools can handle extreme, worst-case contention, where for example millions of users send a payment to one address all at once. There is furthermore a contract-agnostic transformation (RCQ ref) that lets any contract that needs strictly sequential semantics to recover those semantics, as necessary. Concurrent contracts run at arbitrarily high throughput, strictly sequential contracts run at the speed of one CPU (the best we could hope to promise) and neither slows down the other.

Similarly, because every contract modifies state via a set of structured system calls, the rest of a smart contract can be arbitrary, untrusted bytecode. Thruput requires no static analysis or approval process to guarantee commutativity. Like any other blockchain, smart contracts on Thruput are uploaded by untrusted users and may be actively malicious. A bugs in one contract cannot corrupt the rest of the chain.

3 Proposing and Validating a Block

Thruput's state model enables it to execute all transactions in a block in parallel with virtually no synchronization overhead between transactions. This parallelism holds regardless of the frequency that concurrent transactions access overlapping state, and holds regardless of whether Thruput is constructing a new block proposal or validating a proposal from another node.

Executing a transaction proceeds in two phases. First, each transaction starts with an instruction to invoke the contract at some address with some input calldata. A logical thread [3] loads the contract from the database and begins interpreting the RISC-V instructions there. As that proceeds, that contract may sometimes invoke various system calls, which might read additional values from the database or create requests to write values back to the database. A system call may also request to invoke another contract. Contract invocations that fail (i.e. run out of gas or access out of bounds memory) in this phase become no-ops. All writes are buffered locally in that transaction; a transaction can read values that it wrote earlier, but can never read values written by concurrently-running transactions.

Eventually, that top-level contract invocation comes to an end. The second phase of transaction execution is a reserve-commit protocol (like two-phase commit) to apply writes to the database. For each modified key, the transaction first `reserve()`'s the right to apply each modification to the database. In a standard two-phase commit process, this would lock the entire key-value pair. Thruput uses a fine-grained locking scheme, depending on the type of the value. For example, for a token balance (non-negative integer, ref) with snapshot value of 100, Thruput allows up to 100 units worth of subtractions to be `reserve()`d at the same time. If all reservations can be acquired successfully, the transaction then `commit()`s each of its writes.

Nodes in Thruput sometimes are elected leader, to propose a new block, and sometimes must validate a block proposal from another node. When proposing a block, a Thruput node runs the above process and greedily packs in any transactions that succeed at reserving all of their storage updates. Transactions that fail to acquire reservations can be deferred to future blocks. Validation

[3] Each transaction runs as a C++23 coroutine, not a thread, so one thread might run several transactions at once.

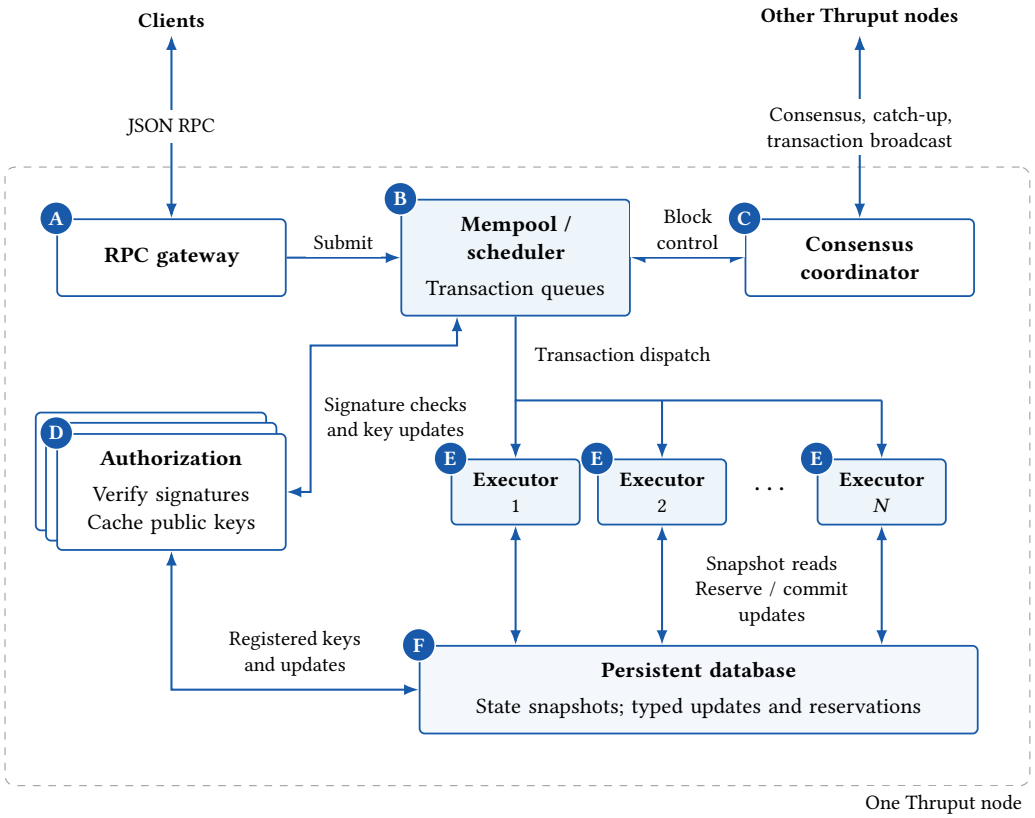


Fig. 2. Data flow within one Thruput logical node.

means executing a defined set of transactions; if any transaction fails that two-phase commit process, then that indicates an invalid proposal, and that block is rejected. Note that the reserve-commit process guarantees that no honest proposer can accidentally produce an invalid block, and guarantees that an honest block proposal satisfies the type-specific invariants for all values in persistent storage.

4 System Architecture

Thruput is built from the ground up to be a distributed system, where the key components can independently scale. Figure 2 outlines the architecture of one node.

An external RPC Gateway (A) mediates access to external clients, who might send transactions via a JSON API or subscribe to a websocket feed of events. Transactions are forwarded to a mempool process (B), which buffers pending transactions locally. A consensus coordinator (C) manages communication with other Thruput nodes, handling consensus messages, requests for past blocks for nodes catching up to the chain, and manages a multicast broadcast subsystem to forward new transactions from the mempool to other Thruput nodes.

Adjacent to the mempool is an authorization service (D), which is responsible for the (expensive) cryptographic signature verification on each transaction. Note that Thruput has no notion of "Externally Owned Account"; instead, every address can configure a set of public keys that can authorize transactions to start with an invocation of the contract at that address. This means

the authorization service must load these registered keys from the persistent database (F). The authorization service is stateless, but maintains a cache of recently used public keys. The mempool stores transactions that pass authorization on a queue of transactions that are ready to execute. When a block of transactions commits, some addresses might have changed their registered public keys, so the database notifies the authorization service and the mempool of all addresses with changed keys that must have their authorization re-checked and cache entries invalidated.

When a Thruput node is called upon to propose a new block, the mempool dispatches transactions from its runqueue to transaction executors (E). These stateless services execute transactions as they arrive, reading and writing to the database as necessary. Thruput's snapshot-read model conveniently allows the executors to trivially cache values read from the database. This design is feasible in large part because of the Thruput execution model. When a transaction reads from the database, that access reads data not in local memory but on a remote machine, incurring nontrivial latency. But during that remote access, the executor's CPU can simply switch to executing another ready transaction, so CPU time is never wasted. In a standard sequential chain, by contrast, that remote access could stall every other transaction in the block.

The mempool prioritizes which transactions to include according to a scheduling policy, related to the number of native coins held at each address and how recently those coins were used to schedule a transaction (see later ref). When a block of transactions commits, the database notifies the mempool of native coin transfers, to inform this scheduling policy.

Transactions may emit events, which are accumulated separately on an event server. When a block commits, these events are forwarded to downstream services, which might power block explorers or other analytical services.

5 Concurrent Data Types

We outline here the data types and APIs which Thruput smart contracts use to interact with persistent state. This is not a complete API reference, and we may add additional types in the future, as needed. We also provide, for example, system calls to emit events, to query what address called the function ('msg.sender'), and to verify a signature, among other functionality.

We design the Thruput semantics to resolve as many conflicts as logically possible by default. However, any contract that needs exclusive access to a particular value can add an exclusive lock flag to any storage write.

5.1 Bytestrings

The minimal, default way for a contract to write data is as an opaque bytestring. This is how most blockchains operate. Yet this gives no semantic information to the Thruput runtime to resolve writes to the same key. Without any information, Thruput cannot do anything but reject transactions that write different values to the same key in a block. By default, Thruput admits pairs of transactions that write the same string to a key. This behavior can be overridden by the exclusive lock flag.

Configuration data, like the address of an administrator on a contract, fits well in this model and has no meaningful notion of commutativity to express to Thruput. One might use the exclusive lock flag when initializing a new contract, to ensure that stateful operations associated with the initialization (like transferring funds) are only performed once. In other settings, such as computing the closing time of an auction and updating a state variable to mark an auction as closed, multiple transactions can make that write and proceed concurrently.

5.2 Nonnegative Integers

Financial applications, like token balances, need some notion of a balance that cannot be negative. Thruput provides an abstracion of a "non-negative integer". The natural API for a token balance

Token Implementation

```
def transfer(to, amount):
    require 0 <= amount <= I128_MAX
    sender = msg.sender
    _adjust(balance_key(sender), -amount)
    _adjust(balance_key(to), amount)
    emit Transfer(sender, to, amount)
    return True

def balanceOf(owner):
    key = balance_key(owner)
    base, delta = nnint_get(key)
    return base + delta

def _adjust(key, change):
    base, delta = nnint_get(key)
    delta += change
    require (base + delta) >= 0
    nnint_set_add(key, base, delta)
```

Pseudocode; a partial ERC20cite implementation

Transfer Velocity Limit

```
def spend(to, amount):
    require msg.sender == owner
    require 0 <= amount <= I128_MAX
    epoch = block_height() // WINDOW

    if bytestring_get("epoch") != epoch:
        bytestring_set("epoch", epoch)
        base, delta = LIMIT, 0
    else:
        base, delta = nnint_get("budget")

    delta -= amount
    require base + delta >= 0
    nnint_set_add("budget", base, delta)
    require token.transfer(to, amount)
```

Pseudocode; a fixed-window spending cap. `owner`, `token`, `LIMIT`, and a positive `WINDOW` are configured at deployment. A missing epoch marker triggers the first reset.

Fig. 3. Two uses of concurrent integer updates. Left: an ERC20-style token interface. Right: a wallet enforcing a transfer limit per window. The runtime checks aggregate balance and budget constraints across transactions, while contracts expose ordinary synchronous methods. Pseudocode omits encodings and arithmetic overflow checks.

would be to add and subtract to it; Thruput gives a generalized version of that API, which we call a `nnint_set_add`(X, Y) operation. X and Y are any two *signed* 128-bit integers. Logically, this operation sets the value of the integer to X, and then adds Y to that base value. The corresponding `nnint_get` API returns the most recently written (X, Y) pair on that key, or the snapshot value (and a 0 adjustment on it) for values not yet modified in that transaction. This API can implement both token balances and more general linear constraints on financial operations, like collateralized borrowing limits and spending velocity constraints.

Contraint: Nonnegativity. Two concurrent transactions that produce `nnint_set_add`(X_1, Y_1) and `nnint_set_add`(X_2, Y_2), respectively, conflict if $X_1 \neq X_2$ (analogously to the bytestring case). A collection of operations `nnint_set_add`$(X, Y_1), \ldots,$ `nnint_set_add`(X, Y_n) is invalid (i.e. reservations of these operations cannot all succeed) if $X + \sum_i^n Y_i < \min(0, X)$; that is, if the value of the integer becomes negative. In the interest of maximimizing flexibility for contracts, we allow $X < 0$ (as contracts can choose to abort before making a `nnint_set_add` system call with such an argument), in which case the constraint means that the value cannot become further negative. Thruput also enforces an analogous constraint against overflowing the maximum value of a 128-bit integer. 128 bits is an arbitrary choice, but we believe provides a reasonable tradeoff between arithmetic width and implementation complexity.

5.2.1 Example: Token Contract. A token contract can represent token balances and allowances as non-negative integers. A transfer means reading a user's snapshot balance, and then outputting an `nnint_set_add`() operation, using the observed balance as the base value. Figure 3 sketches

Borrowing and Repayment

```
def borrow(amount):
    require 0 < amount <= I128_MAX
    user = msg.sender
    _change_debt(user, amount)
    require token.transfer(user, amount)

def repay(amount):
    require 0 < amount <= I128_MAX
    user = msg.sender
    _change_debt(user, -amount)
    require token.transferFrom(
        user, self.address, amount)
```

Shared Borrowing Constraint

```
def _change_debt(user, change):
    dkey = debt_key(user)
    mkey = margin_key(user)

    debt, delta = nnint_get(dkey)
    delta += change

    margin = snapshot_limit(user) - debt

    # Debt cannot be negative
    nnint_set_add(dkey, debt, delta)
    # If margin < 0, debt can only be repaid
    nnint_set_add(mkey, margin, -delta)
```

Snapshot helpers ignore this transaction's pending writes; `delta` accumulates its debt changes. The borrowing limit values collateral using the block's fixed prices and discount factors.

Fig. 4. Collateralized borrowing as a concurrent linear constraint. Interest accrual calculations omitted.

pseudocode for part of an ERC20-style interface. We emphasize that other contracts calling the transfers can call the methods as synchronous cross-contract calls, without knowing how the token is implemented.

5.2.2 Example: Transfer Velocity Limit. The base value portion of the `nnint_set_add()` operations lets the operation extend beyond token balances. A transfer velocity limit, for example, can be implemented as a non-negative integer whose base value resets on a regular frequency.

Figure 3 shows a wallet that permits at most `LIMIT` tokens to be transferred during each window of `WINDOW` blocks. At a window boundary, concurrent calls write the same epoch marker and reset the budget to the same base value; their spending deltas combine. The budget update and token transfer happen within one transaction.

5.2.3 Example: Collateralized Lending. A collateralized borrowing/lending protocol (such as Compound [3], Blend cite, Aave cite) allows users to post collateral (typically some volatile asset) and borrow another asset against it (typically a stable asset, like a USD-pegged stablecoin). The amount that can be borrowed is a complicated calculation which might include pricing the user's posted collateral with various protocol-defined discount factors based on, for example, asset volatility. Interest rates similarly float (typically) in response to a variety of factors, such as the amount of an asset posted as collateral relative to the amount of that asset borrowed.

To implement such a protocol on Thruput, a contract would compute updates to interest rates and collateral valuations once per block. Compound already uses a related pattern on a sequential chain: its interest indices accrue only when the block timestamp advances [2]. Thruput can then implement borrowing headroom and total debt as non-negative integers. Figure 4 sketches a lending market's borrow and repayment paths. The contract computes the borrowing limit available to a user, minus its existing debt, and uses that margin as the base value of the `nnint_set_add()` call.

These loans can become undercollateralized, when, for example, the value of the collateral asset crashes. Each protocol has some mechanism for absorbing those losses; that choice is not especially

relevant here. What matters for Thruput, though, is that these undercollateralized states are still representable, so that contracts can make forward progress to resolve these cases. This setting motivates why we allow the base value of `nnint_set_add()` to be negative.

In other words, Thruput enforces only the constraints that cannot be enforced by a transaction itself. One transaction cannot directly observe concurrent operations from other transactions. Instead, contracts express those invariants to Thruput so that the runtime can enforce them. However, one contract invocation can observe states written in prior blocks, so Thruput places as few constraints as possible on how contracts operate on past state.

5.3 Queues

Some contracts, such as an automated market maker or an orderbook[4], require strictly sequential semantics. For these contracts, Thruput provides a multi-producer, single-consumer queue. This queue allows the contract to recover limited ordering on its own operations, without forcing the whole chain into a single total order.

Constraint: Unique Consumer. A caller of the contract can insert a message into a queue. Insertions do not conflict with each other, and the queue is FIFO-ordered across blocks (and lexicographically sorted within a block). One transaction can then consume entries from the queue one by one, and only one transaction is allowed to consume from the queue in a block. This means that unless the queue is explicitly deleted, every entry that is placed into a queue can be consumed at most once, and entries are never dropped.

5.3.1 Example: AMM Contract. An automated market-maker such as Uniswap [1] maintains reserves of two assets and allows users to exchange either asset for the other, at an exchange rate that adjusts based on the amounts of assets in the AMM. The price calculation formula is different in each implementation, but generally the price changes after every trade; as such, trades generally do not commute with each other, and the AMM needs to execute transactions in a strictly sequential order.

Thruput implements this primitive by wrapping calls to the AMM in a queue. Callers submit trade requests to the queue, and then a separate transaction "turns the crank" to pop elements from the queue and apply them to the AMM. cite people doing this notion of cranking on solana Figure 5 gives a simplified example using Uniswap v2's [1] pricing rule.

There are several limitations to this design. The first is that unlike the rest of Thruput, there is a limit to the number of transactions that a single AMM can handle in a block. This means that an AMM may wish to charge a transaction fee (using e.g. an EIP-1559 style dynamically adjusted fee) to limit the number of enqueued swaps to a rate that can be sequentially processed. In other blockchains, this fee mechanism is handled by the consensus protocol; here, it must be implemented (but the fee revenue can then be paid directly to the liquidity providers of the AMM). Second, calls into an AMM cannot be completed synchronously by other contracts, which limits what can be done within a single transaction.

And finally, synchronous calls that the AMM makes (such as token transfers) might still conflict with other, concurrent transactions. An AMM could handle this by transferring the token sold when the request is enqueued; then, assuming the token contract actually satisfies the token contract specification, the work of processing one trade would not depend on whether the trader still has sufficient funds at the time the request is executed.

[4]A decentralized exchange can be implemented in Thruput's concurrency model, but this requires substantial changes to the trading mechanism [5], which touches on long-running open questions in the economics literature citations

Submit a Swap

```
def swap_x_for_y(amount, min_out):
    require amount > 0 and min_out >= 0
    user = msg.sender

    queue_insert(SWAP_KEY,
        (user, amount, min_out))
```

Process Queued Swaps

```
def crank():
    x, y = bytestring_get(RESERVES)
    amm_address = get_current_address()
    for _ in range(MAX_BATCH):
        request = queue_consume(SWAP_KEY)
        if request is None:
            break
        user, amount, min_out = request

        require token_x.transferFrom(
            user, amm_address, amount)

        # Uniswap v2's 0.3% swap fee
        net = 997 * amount
        out = (net * y) // (1000*x + net)
        if out == 0 or out < min_out:
            # Refund
            require token_x.transfer(
                user, amount)
            continue

        x, y = x + amount, y - out
        require token_y.transfer(user, out)

    bytestring_set(RESERVES, (x, y))
```

Fig. 5. A simple Uniswap v2-style swap operation. Request failure handling is omitted; see §ref

However, that design requires case-by-case analysis and may be difficult to implement in practice. Thruput also introduces a general purpose mechanism, which we call commit locking (§link) to allow contracts to capture a few limited bits of information about concurrent transactions. With commit locking, the queue crank operation is a general purpose mechanism for any sequential contract, which requires no specific assumptions or case by case analysis.

5.4 Replay Nonce Set

6 Addresses and Authorization

Computation on Thruput is so widely available that we have no need for so-called "externally owned accounts"; instead, every user interacts with the chain via a smart wallet. Each address can register one or more public keys, which Thruput uses to authorize transactions that start with an invocation on that address.

This design has the added benefit of not locking Thruput in to any one signature scheme. The current implementation supports only Ed25519 keys, but when we add additional post-quantum signature schemes, users will be able to seamlessly exchange their keys for the new schemes without moving addressees.

Talk about Stellar interop here?

7 Database Design

8 No-Fee Transaction Scheduling

9 Nondeterministic Proposal Data

9.1 Commit Locking

9.2 Retrocausality

9.3 External Calls

10 Related Work

some commentary about CRDTs and how they're insufficient

11 Roadmap

References

[1] Hayden Adams, Noah Zinsmeister, and Dan Robinson. Uniswap v2 Core. https://app.uniswap.org/whitepaper.pdf, 2020.

[2] Compound. Comet: Interest accrual implementation. https://github.com/compound-finance/comet/blob/f766f51583c23acc33b2a7824654ef2029a96804/contracts/CometWithExtendedAssetList.sol#L243-L255. Function `accrueInternal`, revision `f766f51583c2`.

[3] Compound III docs. https://docs.compound.xyz/, 2022.

[4] Rati Gelashvili, Alexander Spiegelman, Zhuolun Xiang, George Danezis, Zekun Li, Dahlia Malkhi, Yu Xia, and Runtian Zhou. Block-stm: Scaling blockchain execution by turning ordering curse to a performance blessing. In *Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*, pages 232–244, 2023.

[5] Geoffrey Ramseyer, Ashish Goel, and David Mazières. {SPEEDEX}: A scalable, parallelizable, and economically efficient decentralized {EXchange}. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 849–875, 2023.